

Testing Graph Database Systems via Equivalent Query Rewriting

Paper Presentation

Qiuyang Mang, Aoyang Fang, Boxi Yu, Hanfei Chen, Pinjia He

ICSE 2024

June 22, 2026

Outline

- 1 Motivation
- 2 Method
- 3 Implementation
- 4 Evaluation
- 5 Takeaways

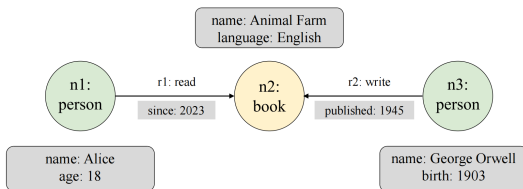
Why Test Graph Databases?

- Graph Database Management Systems (GDBMS) store and query highly connected data.
- Common scenarios include recommendation systems, knowledge graphs, and social networks.
- Bugs may silently corrupt graph traversal results, produce severe slowdowns, or raise wrong exceptions.

Testing challenge

The same logical graph pattern can be expressed in many syntactically different ways.

Labeled Property Graph Model



Core objects

- **Nodes:** entities such as persons and books.
- **Relationships:** typed edges such as read or write.
- **Labels:** categories attached to nodes or edges.
- **Properties:** key-value attributes.

Why it matters

Query correctness depends on both data values and graph traversal semantics.

What Does a GDBMS Logic Bug Look Like?

```
(1) MATCH (A)-[]->(B:L1), (C)<-[]-(A)
    MATCH (C)<-[]-(D:L2)-[]->(D:L2)
    RETURN COUNT(*) -- {7}

(2) MATCH (C)<-[]-(A)-[]->(B:L1)
    MATCH (C)<-[]-(D:L2)-[]->(D:L2)
    RETURN COUNT(*) -- {8}
```

Observation

The two RedisGraph queries express the same graph relationship but return different counts.

Oracle

Equivalent queries should have equivalent behavior.

Why Existing GDBMS Testing Is Hard

Many languages

Cypher, Gremlin, and nGQL use different syntax and query models.

Graph semantics

RDBMS testing oracles mostly target predicates, expressions, and joins rather than graph traversal.

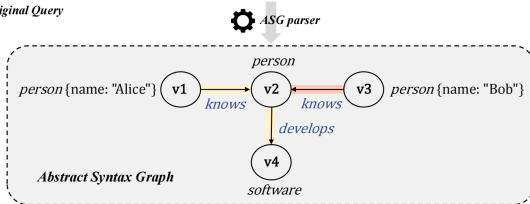
Paper's goal

Build a language-independent metamorphic testing approach that stresses graph data retrieval.

Key Idea: Equivalent Query Rewriting

```
MATCH (n1 : person {name: "Alice"})-[r1 : knows]->(n2 : person), (n3 : person {name : "Bob"})-[r2 : knows]->(n2 : person), (n2 : person)-[r3 : develops]->(n4 : software) RETURN n2
```

Original Query



Equivalent Query

⚙️ Random Walk Covering

```
MATCH (n4 : software)<-[r3 : develops]-(n2 : person)<-[r1 : knows]-(n1 : person {name: "Alice"}), (n2 : person)<-[r2 : knows]-(n3 : person {name : "Bob"}) RETURN n2
```

- ❶ Parse a concrete query into an Abstract Syntax Graph.
- ❷ Cover the graph with different path sequences.
- ❸ Translate each path covering into an equivalent query.
- ❹ Report bugs if behaviors diverge.

Abstract Syntax Graph: Intuition

Definition

Given a query Q , its ASG is a directed graph:

$$G(Q) = (V, E)$$

- V : node categories and their constraints.
- E : relationships that must hold between node categories.

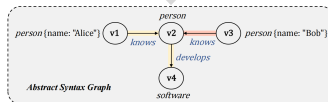
Insight

Query semantics can be embedded into graph paths, independent of the concrete query language.

```
MATCH (n1 : person {name: "Alice"})-[r1 : knows]->(n2 : person), (n3 : person {name: "Bob"})-[r2 : knows]->(n2 : person), (n2 : person)-[r3 : develops]->(n4 : software) RETURN n2
```

Original Query

ASG parser

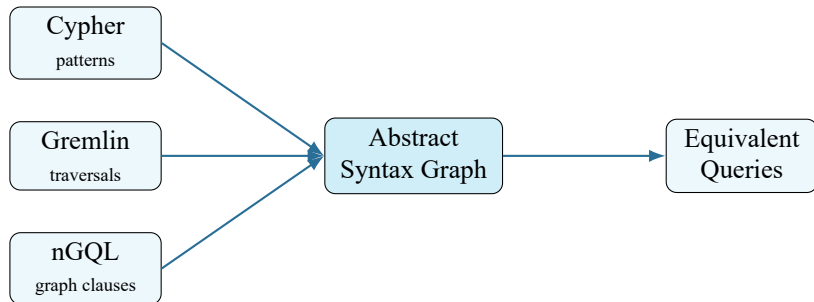


Equivalent Query

Random Walk Covering

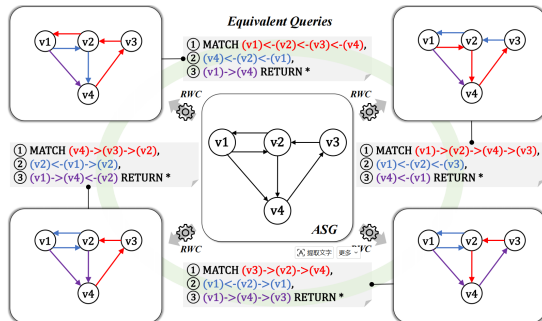
```
MATCH (n4 : software)<-[r3 : develops]-(n2 : person)<-[r1 : knows]-(n1 : person {name: "Alice"}), (n2 : person)<-[r2 : knows]-(n3 : person {name: "Bob"}) RETURN n2
```

ASG Parsing Across Query Languages



- Each new query language mainly needs a translator between query syntax and ASG.
- The paper reports a lightweight Cypher ASG parser of about 260 lines of Python code.
- This abstraction is the reason GRev can test Cypher, Gremlin, and nGQL systems.

Random Walk Covering



Goal

Find a set of non-overlapping paths that collectively cover the ASG.

- Every relationship is covered exactly once.
- Node constraints are distributed across selected paths.
- Different coverings become different equivalent queries.

RWC Algorithm in One Slide

```
Qe = empty query
while ASG still has uncovered content:
    P = randomly select one path
    remove edges covered by P
    assign a subset of constraints to P
    if a vertex is fully covered:
        remove the vertex
    Qe += TransPath2Query(P)
return Qe
```

Invariant

The generated query is equivalent because the selected paths cover all ASG relationships and constraints.

Why random?

Random path choices create many path-covering schemes without handcrafted rewrite rules.

Why Equivalent Queries Trigger Different Plans

```
-- Forward traversal
MATCH (n1:person)-[]->(n2:book)

-- Reversed textual form
MATCH (n2:book)<-[]-(n1:person)
```

- The two queries have the same graph semantics.
- Some GDBMS may choose different start nodes and traversal orders.

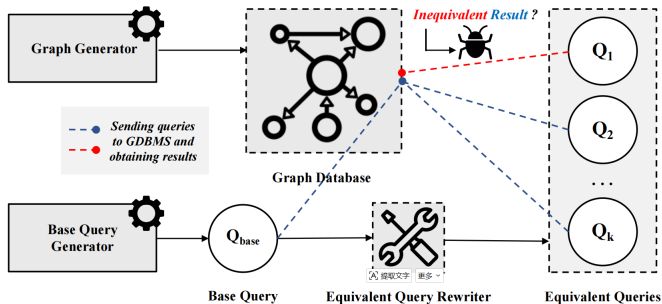
Testing effect

Equivalent query rewriting creates a single-system oracle and stresses query plan generation.

Bug signal

Same semantics, different behavior.

GRev System Overview



- 1 Generate a random labeled property graph.
- 2 Generate a base query Q_{base} .
- 3 Rewrite it into $Q_1, Q_2, \dots, Q_k$.
- 4 Execute and compare behaviors.

Validation Rules

Logic bugs

$R(Q_a) \neq R(Q_b)$ without exceptions.

Exception bugs

Equivalent queries expose unexpected or missing exceptions.

Performance bugs

GRev reports a performance bug when equivalent queries have both:

$$\frac{\max(T_a, T_b) - \min(T_a, T_b)}{\max(T_a, T_b)} \geq 0.8$$

and

$$\max(T_a, T_b) - \min(T_a, T_b) \geq 1000\text{ms.}$$

Experimental Setup

GDBMS	Language	Version	Initial Release
Neo4j	Cypher	5.6.0	2007
RedisGraph	Cypher	6.2.6-v7	2018
MemGraph	Cypher	2.8.0	2017
TinkerGraph	Gremlin	3.6.0	2009
NebulaGraph	nGQL	3.5.0	2019

- Five representative GDBMS across three query languages.
- Experiments ran on Ubuntu 22.04 with Intel i5-13400 and 128 GB memory.
- Research questions evaluate bug-finding effectiveness, test-case quality, and comparison with prior tools.

Main Result: Previously Unknown Bugs

Detected

22

Confirmed

15

Graph-related

14

GDBMS	Detected	Confirmed	Logic	Perf.	Graph-rel.
Neo4j	3	2	0	0	1
RedisGraph	6	2	4	1	4
MemGraph	4	3	3	0	3
TinkerGraph	3	2	2	0	2
NebulaGraph	6	6	3	2	4
Total	22	15	12	3	14

Also found 4 missing exceptions and 3 unexpected exceptions.

Case Study: A RedisGraph Performance Bug

```
(1) MATCH (n0:L3:L2), (n1:L0)
    MATCH (n2:L1)<-[r2:T4]~(n3:L3), (n0:L3)
    WITH n2 MATCH (n4:L3)<-[r3:T3]~(n1), (n0)
    WHERE (n2.k7) RETURN COUNT(*) -- 0.54s

(2) MATCH (n0:L3:L2), (n1:L0)
    MATCH (n2:L1)<-[r2:T4]~(n3:L3), (n0:L3), (n0)
    WITH n2 MATCH (n4:L3)<-[r3:T3]~(n1), (n0)
    WHERE (n2.k7) RETURN COUNT(*) -- 4.82s
```

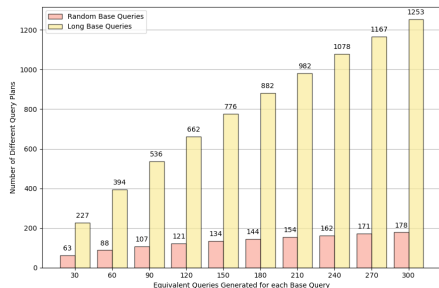
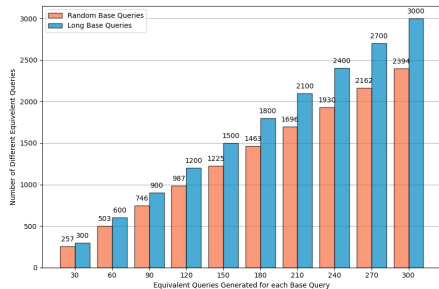
Root cause

The redundant node changes the query plan and causes unnecessary scanning and Cartesian products.

Takeaway

EQR can expose performance bugs inside one GDBMS without cross-system timing comparison.

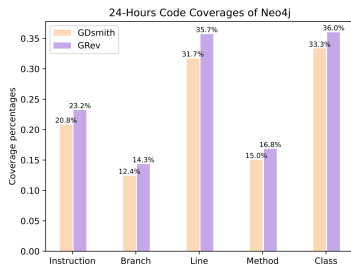
Generated Query and Plan Diversity



- Unique equivalent queries:
Random: 2,394 / 3,000; Long: 3,000 / 3,000.
- Distinct query plans with 300 rewrites per base query: Random: 178; Long: 1,253.
- 1.25M RedisGraph test cases produced 945 bug-triggering cases.
- Average trigger rate: one per 1,322 executions.

Comparison with Existing Techniques

Tool	Approach	Supported Languages	Able to detect shared bugs	Able to test unique features	Insensitive to base query	Detected performance bugs	Detected graph-related bugs
GDsmith	Differential Testing	Cypher	○	○	○	○	●
Grand	Differential Testing	Gremlin	○	○	○	○	○
GDBMeter	Query Partitioning	Cypher, Gremlin	●	●	○	○	○
GRev	Query Rewriting	Cypher, Gremlin, nGQL	●	●	○	●	●



- GRev supports Cypher, Gremlin, and nGQL.
- It can detect graph-related and performance bugs.
- On Neo4j, GRev improves coverage over GDsmith.
- On RedisGraph, GDBMeter has higher coverage, but GRev remains competitive with the same base query generator.

Summary

Problem

GDBMS contain logic, performance, and exception bugs that are hard to detect because the same graph pattern can be expressed in many syntactically different ways.

Method: GRev

- Parse queries into an **Abstract Syntax Graph** (ASG) — language-independent.
- Apply **Random Walk Covering** (RWC) to generate semantically equivalent queries.
- Use equivalent-result divergence as the single-system test oracle.

Results

22 bugs detected (15 confirmed, 14 graph-related) across 5 GDBMS (Neo4j, RedisGraph, MemGraph, TinkerGraph, NebulaGraph) over Cypher, Gremlin, and nGQL.

Significance

First language-independent metamorphic testing approach for GDBMS; exposes unique graph-traversal and performance bugs missed by prior tools.